

LAPORAN PRAKTIKUM MONGODB DENGAN PYTHON



Dosen Pengampu : M. Harry Khomas Saputra S.T, M.TI.

Kelompok 5

Alexander Dwiky P. P.	225443003
Fahmi Hadi Firdaus	225443007
Fatih Akmal Alqathani	225443009
Muhammad Izzudun Irsyad	225443018
Muhammad Zaidan Habibi	225443020
Naufal Rabbani	225443022

**TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI
POLITEKNIK MANUFAKTUR BANDUNG
TAHUN AJARAN 2025/2026**

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan praktikum basis data ini tepat pada waktunya. Laporan ini disusun untuk memenuhi tugas mata kuliah dengan fokus materi pada implementasi basis data NoSQL menggunakan MongoDB, yang mencakup teknik otomatisasi data (data ingestion) dengan Python, pemrosesan analisis pabrik menggunakan fitur Aggregation Framework, hingga pemahaman struktur desain JSON Schema.

Penulis mengucapkan terima kasih kepada semua pihak yang telah membantu dalam pelaksanaan praktikum dan penyusunan laporan ini, khususnya kepada:

1. Bapak M. Harry Khomas Saputra selaku dosen pengampu yang sudah membimbing dalam pembelajaran mata kuliah Praktikum Basis Data.
2. Ibu Anis Rosiani selaku PLP Mata Kuliah Praktikum Basis Data yang telah mendampingi dan membantu dalam pelaksanaan praktikum.
3. Ibu Siti Aminah selaku Kepala Program Studi Teknologi Rekayasa Informatika yang telah memberikan fasilitas dan dukungan dalam kegiatan praktikum.
4. Semua pihak terutama rekan-rekan yang tidak dapat disebutkan satu per satu yang telah membantu dalam penyelesaian laporan ini.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan untuk perbaikan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi yang berguna dalam mempelajari MongoDB dan basis data NoSQL.

Bandung, 06 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	2
DAFTAR ISI	3
BAB I PENDAHULUAN	5
1.1 Latar Belakang	5
1.2 Tujuan Praktikum	6
BAB II TUGAS TERSTRUKTUR.....	7
2.1 Persiapan Lingkungann.....	7
2.2 Struktur Program.....	8
2.3 Menu 1: Input Data Produksi Baru	9
2.4 Menu 2: Menampilkan Data Per Mesin	10
2.7 Menu 3: Hitung Reject Rate >5%.....	11
2.6 Menu 4: Ekspor Laporan Bulanan (CSV).....	12
2.7 Kesimpulan	13
BAB III STUDI KASUS.....	14
3.1 Generator Data Dummy	14
3.2 Skrip Uji Coba: Data 1 Jam Terakhir	16
3.3 Skrip Uji Coba: Menghitung Rata-rata Suhu	16
3.5 Skrip Uji Coba: Mencari Suhu Bahaya (>90°C)	17
3.6 Skrip Utama : Aplikasi Monitoring	18
3.6 Kesimpulan	20
BAB IV TUGAS KELOMPOK.....	21
4.1 Studi Kasus	22
4.2 Membuat Koneksi	22

4.3 Generate Data Dummy (Publikasi).....	23
4.4 Menyimpan ke Database (Subscribe)	24
4.5 Membuat Analisis	27
4.6 Kesimpulan	30
BAB V PENUTUP	31

BAB I

PENDAHULUAN

1.1 Latar Belakang

Di era industri 4.0 setiap mesin dan sensor di pabrik terus-menerus menghasilkan banyak data penting, seperti suhu mesin dan catatan produksi. Jika tidak dikelola dengan baik, tumpukan informasi ini hanya akan menguap dan terbangung percuma. Oleh karena itu, dibutuhkan sebuah sistem yang mampu menangkap, menyimpan, dan menganalisis data tersebut secara otomatis. Bahasa pemrograman Python dan database MongoDB dipilih sebagai solusi utama karena keduanya sangat fleksibel dan cocok untuk menangani aliran data mesin yang masuk secara terus-menerus.

Untuk mengirimkan data dari mesin di lapangan ke server pusat, dibutuhkan jalur komunikasi khusus yang ringan dan cepat. Sistem ini menggunakan protokol komunikasi MQTT yang memang dirancang khusus untuk menghubungkan perangkat-perangkat Internet of Things (IoT) di lingkungan industri. Melalui praktikum ini, penggabungan antara Python, MongoDB, dan MQTT akan dipraktikkan langsung untuk membangun sebuah simulasi sistem pemantauan data pabrik yang utuh. Hasilnya, sistem diharapkan tidak hanya mampu merekam riwayat data secara otomatis, tetapi juga bisa memberikan laporan dan peringatan kondisi pabrik secara real-time.

1.2 Tujuan Praktikum

Tujuan yang ingin dicapai dalam pelaksanaan praktikum ini adalah sebagai berikut:

- a. Menginstal dan mengatur virtual environment Python beserta library pendukung industri (pymongo, pandas, python-dotenv, matplotlib, dan paho-mqtt)
- b. Membangun Koneksi Basis Data yang Aman dengan menghubungkan aplikasi Python ke server MongoDB menggunakan environment variable
- c. Mengeksekusi Operasi CRUD pada dokumen berformat BSON di MongoDB secara terprogram menggunakan driver PyMongo.
- d. Mengimplementasikan Aggregation Pipeline untuk menyaring, mengelompokkan, dan menghitung ringkasan statistik data secara efisien.

BAB II

TUGAS TERSTRUKTUR

2.1 Persiapan Lingkungann

Menyiapkan file .env untuk koneksi ke database

Program:

```
MONGO_URI= mongodb://localhost:27017
DB_NAME= latihan_terstruktur_kelompok5
```

Koneksi ke database

Program:

```
import os
import logging
import pandas as pd
from datetime import datetime
from pymongo import MongoClient
from dotenv import load_dotenv

load_dotenv()

# Koneksi ke MongoDB
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]

# Set up logging
logging.basicConfig(
    filename='app.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
```

Bagian ini adalah fondasi utama dari aplikasi. Di sini, program mengundang semua library pembantu seperti pandas untuk tabel dan pymongo untuk database. Kode ini membaca alamat rahasia MongoDB dari file .env agar lebih aman, lalu membuka koneksinya.

2.2 Struktur Program

Program:

```
# Menu utama
def menu ():
    while True:
        print("==== Menu =====")
        print("1. Input Data")
        print("2. Tampilkan Per Mesin")
        print("3. Hitung Reject Rate")
        print("4. Export Laporan Bulanan")
        print("5. Keluar")

        pilihan = input("Pilih menu: ")
        if pilihan == "1":
            input_data()
        elif pilihan == "2":
            tampilkan_data_mesin()
        elif pilihan == "3":
            hitung_reject_rate()
        elif pilihan == "4":
            export_laporan()
        elif pilihan == "5":
            print("Keluar dari program.")
            break
        else:
            print("Pilihan tidak valid. Silakan coba lagi.\n")

# Jalankan program
if __name__ == "__main__":
    menu()
```

Output:

```
(venv) PS C:\Users\Akmal\praktikum6> & c:\Users\Akmal\praktikum6\venv\Scripts\python.exe c:/Users/Akmal/praktikum6/app_produksi.py
==== Menu =====
1. Input Data
2. Tampilkan Per Mesin
3. Hitung Reject Rate
4. Export Laporan Bulanan
5. Keluar
Pilih menu: []
```

Program menggunakan perulangan while True agar menu terus muncul di layar terminal dan tidak langsung mati setelah satu tugas selesai. Ketika pengguna mengetikkan angka pilihan (1 sampai 5), program akan memanggil fungsi spesifik yang sudah disiapkan.

2.3 Menu 1: Input Data Produksi Baru

Program:

```
import os
from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]

pipeline = [
    {"$group": {
        "_id": "$mesin",
        "rata_suhu": {"$avg": "$suhu"},
        "max_getaran": {"$max": "$getaran"},
        "count": {"$sum": 1}
    }},
    {"$sort": {"rata_suhu": -1}}
]

results = collection.aggregate(pipeline)
print("Rata-rata suhu per mesin:")
for doc in results:
    print(f"{doc['_id']} -> rata:{doc['rata_suhu']:.2f}°C, max getaran:{doc['max_getaran']}, jumlah data:{doc['count']}")

client.close()
```

Output:

```
PS C:\Users\Akmal\praktikum6> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Akmal\praktikum6\venv\Scripts\Activate.ps1)
(venv) PS C:\Users\Akmal\praktikum6> & c:\Users\Akmal\praktikum6\venv\Scripts\python.exe c:/Users/Akmal/praktikum6/app_produk...
===== Menu =====
1. Input Data
2. Tampilkan Per Mesin
3. Hitung Reject Rate
4. Export Laporan Bulanan
5. Keluar
Pilih menu: 1
Batch: B002
Mesin: CNC-01
Jumlah: 160
Reject: 10
Tanggal (YYYY-MM-DD): 2026-05-06
Data berhasil disimpan.
```

Fungsi ini bertugas mengumpulkan data ketikan dari pengguna satu per satu. Data tersebut kemudian dibungkus ke dalam format dictionary. Khusus untuk tanggal, format teks diubah menjadi format waktu resmi (datetime) agar bisa diolah nanti. Setelah dibungkus, data langsung ditembakkan ke MongoDB menggunakan perintah `insert_one()`.

2.4 Menu 2: Menampilkan Data Per Mesin

Program:

```
# Tampilan Data per mesin
def tampilkan_data_mesin():
    try:
        mesin = input("Masukkan nama mesin:")
        cursor = db.produksi.find({"mesin": mesin}, {"_id": 0})
        df = pd.DataFrame(list(cursor))

        if df.empty:
            print("Tidak ada data.\n")
        else:
            print(df, "\n")

        logging.info(f"Query data mesin: {mesin}")

    except Exception as e:
        logging.error(f"Error tampil mesin: {e}")
```

Output:

```
===== Menu =====
1. Input Data
2. Tampilkan Per Mesin
3. Hitung Reject Rate
4. Export Laporan Bulanan
5. Keluar
Pilih menu: 2
Masukkan nama mesin:CNC-01
  batch  mesin  jumlah  reject  tanggal
0  B002  CNC-01     160      10  2026-05-06
```

Fungsi ini meminta pengguna mengetikkan nama mesin yang ingin dicari. Program lalu menyuruh MongoDB mencari semua data yang cocok dengan nama mesin tersebut menggunakan perintah `find()`. Agar tampilannya tidak berantakan, data mentah dari MongoDB tersebut dimasukkan ke dalam DataFrame milik Pandas sehingga muncul di layar dalam bentuk tabel yang rapi.

2.7 Menu 3: Hitung Reject Rate >5%

Program:

```
# Menghitung Reject Rate
def hitung_reject_rate():
    try:
        df = pd.DataFrame(list(db.produksi.aggregate([1
            {
                "$project": {
                    "batch": 1,
                    "mesin": 1,
                    "reject_rate": {
                        "$multiply": [{"$divide":
                            ["$reject", "$jumlah"]}, 100]
                    }
                },
            },
            {"$match": {
                "reject_rate": {"$gt": 5}
            }}
        ])))

        print(df if not df.empty else "Tidak ada data\n")
        logging.info("Hitung reject rate")

    except Exception as e:
        logging.error(f"Error hitung reject rate: {e}")
```

Hasil:

```
===== Menu =====
1. Input Data
2. Tampilkan Per Mesin
3. Hitung Reject Rate
4. Export Laporan Bulanan
5. Keluar
Pilih menu: 3

   _id batch  mesin reject_rate
0  69fb04254b8d7b21b177b70e  B002  CNC-01      6.250000
1  69fb05e04b8d7b21b177b70f  B002  CNC-02     11.764706
```

Fungsi ini adalah fitur analitik yang pintar. Alih-alih menghitung secara manual di Python, program menggunakan fitur Aggregation MongoDB untuk melakukan perhitungan matematika langsung di dalam database. Program membagi jumlah reject dengan total jumlah, lalu dikali 100. Setelah itu, tahap \$match akan menyaring dan hanya meloloskan batch mesin yang angka kecacatannya lebih dari 5%.

2.6 Menu 4: Ekspor Laporan Bulanan (CSV)

Program:

```
# export data ke CSV bulanan
def export_laporan():
    try:
        bulan_tahun = input("Masukkan bulan dan tahun (MM-YYYY): ")
        bulan, tahun = bulan_tahun.split("-")
        df = pd.DataFrame(list(db.produksi.aggregate([
            {"$match": {
                "$expr": {
                    "$and": [
                        {"$eq": [{"month": "$tanggal"}, int(bulan)]},
                        {"$eq": [{"year": "$tanggal"}, int(tahun)]}
                    ]
                }
            },
            {
                "$group": {
                    "_id": "$mesin",
                    "total_jumlah": {"$sum": "$jumlah"},
                    "total_reject": {"$sum": "$reject"}
                }
            },
            {
                "$project": {
                    "_id": 0,
                    "mesin": "$_id",
                    "total_jumlah": 1,
                    "total_reject": 1
                }
            }
        ])))

        filename = f"Laporan_{bulan_tahun}.csv"
        df.to_csv(filename, index=False)

        print(f"Laporan berhasil diekspor ke {filename}\n")
        logging.info(f"Export laporan: {filename}")

    except Exception as e:
        logging.error(f"Error export laporan: {e}")
```

Output:

```
Menu
1. Input Data
2. Tampilkan Per Mesin
3. Hitung Reject Rate
4. Export Laporan Bulanan
5. Keluar
Pilih menu: 4
Masukkan bulan dan tahun (MM-YYYY): 05-2026
Laporan berhasil diekspor ke Laporan_05-2026.csv

Laporan_05-2026.csv
1 total_jumlah,total_reject,mesin
2 170,20,CNC-02
3 160,10,CNC-01
4 |
```

Fungsi ini bertugas membuat rekap laporan bulanan. Pertama, program memotong input pengguna untuk mendapatkan angka bulan dan tahun. Lalu, dengan Aggregation Pipeline, program menyaring data yang cocok dengan bulan dan tahun tersebut, dan menjumlahkan (sum) total produksi beserta total reject untuk masing-masing mesin. Hasil akhir berupa tabel Pandas kemudian diubah menjadi file Excel/CSV menggunakan perintah `to_csv()`.

2.7 Kesimpulan

Program berhasil mengamankan setiap proses dari potensi crash (berhenti mendadak) akibat salah input melalui penerapan sistem penangkap error (try-except). Seluruh alur eksekusi mulai dari pengiriman data baru, pencarian data spesifik, hingga perhitungan persentase kecacatan (aggregation) berhasil diproses secara cepat dan akurat langsung di sisi server MongoDB. Selain itu, eksekusi program juga terbukti terintegrasi dengan mulus pada sistem komputer lokal, yang ditunjukkan oleh keberhasilan program dalam merekam setiap aktivitas operasi ke dalam file log secara real-time dan mencetak hasil laporan agregasi menjadi file berekstensi CSV secara otomatis.

BAB III

STUDI KASUS

Pada studi kasus ini, mahasiswa berperan sebagai seorang engineer yang ditugaskan untuk membangun sebuah dashboard monitoring berbasis teks (konsol) bagi manajer pabrik. Aplikasi pemantauan yang diberi nama `monitor.py` ini dirancang menggunakan database `studi_kasus` dan koleksi `suhu_mesin` untuk mengolah dan menganalisis aliran data sensor yang terjadi dalam kurun waktu satu jam terakhir. Secara garis besar, aplikasi ini dituntut untuk memiliki kemampuan sebagai berikut:

1. Menampilkan informasi suhu paling terkini dari setiap mesin.
2. Melakukan perhitungan rata-rata suhu pada masing-masing mesin menggunakan fitur agregasi.
3. Mendeteksi dan menampilkan peringatan untuk mesin yang mengalami suhu kritis (di atas 90°C).
4. Mencatat riwayat peringatan bahaya (alarm) secara otomatis ke dalam file eksternal `alarm.log`.
5. Memiliki sistem penanganan error (error handling) yang baik, di mana program tidak akan crash, melainkan menampilkan pesan status "OFFLINE" apabila koneksi ke database terputus.
6. Dilengkapi dengan panduan penggunaan program di dalam file `README.txt`.

3.1 Generator Data Dummy

Program:

```
from dotenv import load_dotenv
import random
from datetime import datetime, timedelta
from pymongo import MongoClient
import os

load_dotenv()

client = MongoClient("mongodb://localhost:27017/")
db = client[os.getenv("DB_NAME")]
col = db["suhu_mesin"]
```

```

machines = ["M01", "M02", "M03", "M04"]

def generate_data():
    return {
        "machine_id": random.choice(machines),
        "timestamp": datetime.now() - timedelta(minutes=random.randint(0,120)),
        "temperature": round(random.uniform(60, 100), 2),
        "distance": round(random.uniform(0, 200), 2),
        "light_intensity": random.randint(0, 1000),
        "vibration": round(random.uniform(0, 5), 2),
        "infrared_detected": random.choice([True, False]),
        "ultrasonic_distance": round(random.uniform(2, 400), 2)
    }

for _ in range(100):
    data = generate_data()
    col.insert_one(data)

```

Output:

```

(venv) PS C:\Users\Akmal\praktikum6> & c:\Users\Akmal\praktikum6\venv\Scripts\python.exe c:/Users/Akmal/praktikum6/studkas_dummy.py
(venv) PS C:\Users\Akmal\praktikum6>

```

★ suhu_mesin					
	_id ObjectId	machine_id String	timestamp Date	temperature Double	distance Double
1	ObjectId('69fc02fd7311503...	"M02"	2026-05-07T08:48:57.933+0...	74.56	102.49
2	ObjectId('69fc02fe7311503...	"M03"	2026-05-07T08:23:58.417+0...	95.17	131.32
3	ObjectId('69fc02fe7311503...	"M02"	2026-05-07T08:24:58.419+0...	74.79	106.9
4	ObjectId('69fc02fe7311503...	"M04"	2026-05-07T08:38:58.421+0...	97.99	170.02
5	ObjectId('69fc02fe7311503...	"M04"	2026-05-07T08:22:58.518+0...	86.3	61.14
6	ObjectId('69fc02fe7311503...	"M04"	2026-05-07T09:59:58.520+0...	68.41	119.25
7	ObjectId('69fc02fe7311503...	"M02"	2026-05-07T09:02:58.523+0...	73.19	39.68
8	ObjectId('69fc02fe7311503...	"M02"	2026-05-07T08:33:58.525+0...	68.82	3.57
9	ObjectId('69fc02fe7311503...	"M02"	2026-05-07T09:52:58.530+0...	96.8	143.15
10	ObjectId('69fc02fe7311503...	"M03"	2026-05-07T09:12:58.532+0...	83.11	62.99
11	ObjectId('69fc02fe7311503...	"M02"	2026-05-07T09:11:58.534+0...	75.17	195.24
12	ObjectId('69fc02fe7311503...	"M02"	2026-05-07T08:35:58.536+0...	62.52	134.5
13	ObjectId('69fc02fe7311503...	"M01"	2026-05-07T08:58:58.540+0...	83.89	96.58
14	ObjectId('69fc02fe7311503...	"M04"	2026-05-07T09:15:58.542+0...	60.35	49.08
15	ObjectId('69fc02fe7311503...	"M01"	2026-05-07T09:58:58.543+0...	94.99	183.65
16	ObjectId('69fc02fe7311503...	"M03"	2026-05-07T08:55:58.544+0...	97.25	50.56
17	ObjectId('69fc02fe7311503...	"M03"	2026-05-07T09:42:58.547+0...	98.33	160.76
18	ObjectId('69fc02fe7311503...	"M04"	2026-05-07T08:44:58.548+0...	88.1	185.6

Program ini berfungsi untuk menciptakan 100 data sensor tiruan secara acak dan langsung menyimpannya ke dalam database MongoDB. Data ini dibutuhkan sebagai bahan uji coba agar kita memiliki riwayat aktivitas mesin yang bisa dianalisis nanti.

3.2 Skrip Uji Coba: Data 1 Jam Terakhir

Program:

```
from datetime import datetime, timedelta
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["Studikasukelompok5"]
col = db["suhu_mesin"]

one_hour_ago = datetime.now() - timedelta(hours=1)

data_1_jam = list(col.find({
    "timestamp": { "$gte": one_hour_ago }
}))

for d in data_1_jam:
    print("\n", d)
```

Output:

```
(venv) PS C:\Users\Akmal\praktikum6> & c:\Users\Akmal\praktikum6\venv\Scripts\python.exe c:/Users/Akmal/praktikum6/studkas_1jam.py
(venv) PS C:\Users\Akmal\praktikum6> █
```

Program ini berperan sebagai uji coba dasar untuk menyaring data riwayat mesin yang hanya terjadi dalam kurun waktu satu jam terakhir. Program menggunakan perhitungan waktu mundur (timedelta) untuk menentukan batas waktu pencarian di database.

3.3 Skrip Uji Coba: Menghitung Rata-rata Suhu

Program:

```
from datetime import datetime, timedelta
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["Studikasukelompok5"]
col = db["suhu_mesin"]

one_hour_ago = datetime.utcnow() - timedelta(hours=1)

pipeline = [
    {
        "$match": {
            "timestamp": { "$gte": one_hour_ago }
        }
    },
    {
        "$group": {
            "_id": "$machine_id",
            "avg_temperature": { "$avg": "$temperature" }
        }
    },
    {
        "$sort": { "_id": 1 }
    }
]
```



```
]

result = list(col.aggregate(pipeline))

for r in result:

    print(f"Mesin: {r['_id']}, Rata-rata suhu: {r['avg_temperature']:.2f} °C")
```

Output:

```
(venv) PS C:\Users\Akmal\praktikum6> & c:\Users\Akmal\praktikum6\venv\Scripts\python.exe c:/Users/Akmal/praktikum6/studkas_rata2suhu.py
c:\Users\Akmal\praktikum6\studkas_rata2suhu.py:8: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  one_hour_ago = datetime.utcnow() - timedelta(hours=1)
(venv) PS C:\Users\Akmal\praktikum6> █
```

Skrip ini menggunakan fitur agregasi MongoDB untuk mengelompokkan data berdasarkan nama mesin. Setelah dikelompokkan, program secara otomatis menghitung nilai rata-rata suhu dari setiap mesin tersebut.

3.5 Skrip Uji Coba: Mencari Suhu Bahaya (>90°C)

Program:

```
from datetime import datetime, timedelta
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["Studikases_kelompok5"]
col = db["suhu_mesin"]

one_hour_ago = datetime.now() - timedelta(hours=1)

pipeline = [
    {
        "$match": {
            "timestamp": { "$gte": one_hour_ago }
        }
    },
    {
        "$group": {
            "_id": "$machine_id",
            "max_temperature": { "$max": "$temperature" }
        }
    },
    {
        "$match": {
            "max_temperature": { "$gt": 90 }
        }
    }
]

result = list(col.aggregate(pipeline))

for r in result:
    print(f"Mesin: {r['_id']} | Suhu maksimum: {r['max_temperature']} °C")
```

Output:

```
PS C:\Users\Akmal\praktikum6> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Akmal\praktikum6\venv\Scripts\Activate.ps1)
(venv) PS C:\Users\Akmal\praktikum6> & c:\Users\Akmal\praktikum6\venv\Scripts\python.exe c:/Users/Akmal/praktikum6/studkas_suhubahaya.py
(venv) PS C:\Users\Akmal\praktikum6>
```

Program ini bertugas khusus untuk mencari mesin mana yang pernah mengalami panas di atas 90°C. Sistem akan memilih suhu paling tinggi (max) pada setiap mesin, lalu membuang data mesin yang suhunya masih dalam batas aman.

3.6 Skrip Utama : Aplikasi Monitoring

Program:

```
from datetime import datetime, timedelta, UTC
from pymongo import MongoClient, errors

try:
    client = MongoClient("mongodb://localhost:27017/", serverSelectionTimeoutMS=3000)
    client.admin.command("ping")

    db = client["Studikasukelompok5"]
    col = db["suhu_mesin"]

    one_hour_ago = datetime.now(UTC) - timedelta(hours=1)

    latest_pipeline = [
        {"$sort": {"machine_id": 1, "timestamp": -1}},
        {
            "$group": {
                "_id": "$machine_id",
                "latest_temp": {"$first": "$temperature"},
                "latest_time": {"$first": "$timestamp"}
            }
        },
        {"$sort": {"_id": 1}}
    ]
    latest_data = list(col.aggregate(latest_pipeline))

    alarm_pipeline = [
        {"$match": {"timestamp": {"$gte": one_hour_ago}}},
        {
            "$group": {
                "_id": "$machine_id",
                "max_temp": {"$max": "$temperature"}
            }
        },
        {"$match": {"max_temp": {"$gt": 90}}}
    ]
    alarm_data = list(col.aggregate(alarm_pipeline))

    avg_pipeline = [
        {"$match": {"timestamp": {"$gte": one_hour_ago}}},
        {
            "$group": {
                "_id": "$machine_id",
```

```

        "avg_temp": {"$avg": "$temperature"}
    }
},
{"$sort": {"_id": 1}}
]
avg_data = list(col.aggregate(avg_pipeline))

print("\n=== SUHU TERKINI PER MESIN ===")
for d in latest_data:
    print(f"Mesin {d['_id']} | Suhu: {d['latest_temp']:.2f}°C | Waktu: {d['latest_time']}")

print("\n=== MESIN DENGAN ALARM (>90°C) ===")
if not alarm_data:
    print("Tidak ada mesin yang memicu alarm")
else:
    for d in alarm_data:
        print(f"Mesin {d['_id']} | Suhu maksimum: {d['max_temp']:.2f}°C")

print("\n=== RATA-RATA SUHU PER MESIN ===")
for d in avg_data:
    print(f"Mesin {d['_id']} | Rata-rata suhu: {d['avg_temp']:.2f}°C")

if alarm_data:
    with open("alarm.log", "a") as f:
        for d in alarm_data:
            log_line = (
                f"[{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}] "
                f"Mesin={d['_id']} | MaxTemp={d['max_temp']:.2f}°C\n"
            )
            f.write(log_line)

except errors.ServerSelectionTimeoutError:
    print("Sistem monitoring OFFLINE: MongoDB tidak dapat dihubungi.")
except Exception:
    print("Terjadi gangguan pada sistem monitoring.")

```

Output:

```

(venv) PS C:\Users\Akmal\praktikum6> py studkas_app.py

=== SUHU TERKINI PER MESIN ===
Mesin M01 | Suhu: 79.10°C | Waktu: 2026-05-07 10:05:58.566000
Mesin M02 | Suhu: 92.52°C | Waktu: 2026-05-07 10:11:58.576000
Mesin M03 | Suhu: 81.04°C | Waktu: 2026-05-07 10:10:58.595000
Mesin M04 | Suhu: 75.64°C | Waktu: 2026-05-07 10:05:58.603000

=== MESIN DENGAN ALARM (>90°C) ===
Mesin M01 | Suhu maksimum: 99.36°C
Mesin M04 | Suhu maksimum: 99.79°C
Mesin M02 | Suhu maksimum: 96.80°C
Mesin M03 | Suhu maksimum: 98.99°C

=== RATA-RATA SUHU PER MESIN ===
Mesin M01 | Rata-rata suhu: 81.35°C
Mesin M02 | Rata-rata suhu: 75.11°C
Mesin M03 | Rata-rata suhu: 83.52°C
Mesin M04 | Rata-rata suhu: 82.52°C
(venv) PS C:\Users\Akmal\praktikum6>

```

Ini adalah program final yang menggabungkan seluruh fitur pemantauan (suhu terkini, suhu rata-rata, dan alarm bahaya) menjadi satu dashboard informasi di layar terminal. Selain menampilkan ringkasan data, program ini juga otomatis mencatat riwayat mesin yang kepanasan ke dalam file alarm.log dan memiliki sistem peringatan jika database tiba-tiba mati.

3.6 Kesimpulan

Berdasarkan hasil eksekusi programnya, aplikasi dashboard monitoring ini terbukti mampu berjalan dengan sangat lancar dan stabil. Program secara otomatis berhasil menarik kumpulan data dari satu jam terakhir, melakukan perhitungan matematis menggunakan fitur agregasi MongoDB di sisi server, dan menyajikan hasilnya menjadi sebuah laporan informasi yang rapi di layar terminal. Selain fungsionalitas pembacaan datanya yang baik, sistem ini juga sangat tangguh karena mampu mencatat riwayat mesin yang kepanasan ke dalam file log secara mandiri, serta dilengkapi dengan sistem pelindung error yang cerdas sehingga program tidak akan mendadak mati (crash), melainkan akan memunculkan peringatan "OFFLINE" dengan aman apabila koneksi ke database terputus.

BAB IV

TUGAS KELOMPOK

Tugas ini mencakup sistem pemantauan yang dibaca oleh sensor dan dikelola agar data – data mentah ini dapat ditampilkan menjadi informasi yang berguna untuk user. Mahasiswa diberikan tugas untuk membuat proyek terkait dan kolaboratif dengan arsitektur IoT berbasis MQTT. Beberapa anggota dibagi tugas menjadi:

1. Anggota 1 – MQTT Publisher Simulasi

Membuat skrip `pub_udara.py` untuk mensimulasikan 10 sensor kualitas udara dan mengirim data sensor ke topik MQTT pabrik/udara secara berkala setiap 10 detik, termasuk simulasi kondisi berbahaya pada nilai PM2.5.

2. Anggota 2 – MQTT Subscriber dan Penyimpanan Data

Membuat skrip `sub_udara.py` untuk menerima data dari MQTT, menyimpan data ke MongoDB, mendeteksi anomali PM2.5 berbahaya, serta memberikan notifikasi dan menyimpan alert ke koleksi khusus.

3. Anggota 3 – Analisis Data dan Dashboard

Membuat skrip `analisis_dashboard.py` untuk mengambil data 24 jam terakhir dari MongoDB, menghitung rata-rata PM2.5 per lokasi menggunakan agregasi, menampilkan grafik dashboard, dan mengekspor data ke file CSV.

4. Anggota 4 – Status Sensor dan Dokumentasi

Membuat skrip `status_udara.py` untuk menampilkan status terkini setiap sensor berdasarkan data MQTT atau MongoDB, serta menyusun dokumentasi proyek seperti README, laporan PDF, dan persiapan presentasi kelompok.

4.1 Studi Kasus

Sebuah greenhouse tanaman tomat menggunakan sistem monitoring berbasis IoT untuk memantau kondisi lingkungan secara real-time. Sistem ini memakai sensor cahaya dengan satuan foot-candle (fc) untuk mendeteksi kondisi terlalu terang atau redup, sensor kelembaban tanah untuk mengetahui kebutuhan penyiraman, serta sensor kualitas udara untuk memantau kondisi udara di sekitar tanaman. Data dari seluruh sensor dikirim melalui MQTT dan disimpan ke MongoDB agar dapat dianalisis dan ditampilkan pada dashboard monitoring sehingga petani dapat menjaga kondisi tanaman tomat tetap optimal.

4.2 Membuat Koneksi

```
import os
from dotenv import load_dotenv
import paho.mqtt.client as mqtt
from pymongo import MongoClient

load_dotenv()

def get_mqtt_client():

    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    client.username_pw_set(
        os.getenv("MQTT_USER"),
        os.getenv("MQTT_PASS")
    )
    return client

def get_mongo_client():
    mqtt_user = os.getenv("MQTT_USER")
    mqtt_pass = os.getenv("MQTT_PASS")
    mqtt_broker = os.getenv("MQTT_BROKER")

    mongo_uri = f"mongodb://{mqtt_user}:{mqtt_pass}@{mqtt_broker}:948/"
    return MongoClient(mongo_uri)
```

4.3 Generate Data Dummy (Publikasi)

Publikasi berperan sebagai pengirim data, tetapi tidak boleh langsung terhubung dengan database, melainkan data harus disimpan di broker terlebih dahulu.

```
import time
import random
import json
from datetime import datetime
import paho.mqtt.client as mqtt
from pymongo import MongoClient
from koneksi import get_mqtt_client
#from enkripsi import encrypt_data

TOPIC = "sensor5"

client = get_mqtt_client()

lokasi = ["Lokasi A", "Lokasi B", "Lokasi C", "Lokasi D", "Lokasi E"]

#generate data
def generate_data( lokasi):

    if random.random() < 0.10:
        suhu = round(random.choice([
            random.uniform(0, 21),
            random.uniform(29, 40)
        ]), 2)
    else:
        suhu = round(random.uniform(22, 29), 2)

    if random.random() < 0.05:
        pm25 = random.randint(50, 100)
    else:
        pm25 = random.randint(5, 50)

    return{
        "lokasi":random.choice(lokasi),
        "suhu": suhu,
        "cahaya": random.randint(900, 10000),
        "kelembapan_udara": random.randint(0, 100),
        "kelembapan_tanah": random.randint(0, 100),
        "pm25":pm25
    }
```

```

while True:
    data = generate_data(lokalasi)
    payload = json.dumps(data)
    #pub enkripsi payload = encrypt_data(data)

    client.publish(TOPIC,payload)
    print(f"\n[{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}] Data terkirim ke topic '{TOPIC}':")
    print(f" Lokasi      : {data['lokasi']}")
    print(f" Suhu          : {data['suhu']} °C")
    print(f" Cahaya        : {data['cahaya']} fc")
    print(f" Kelembapan Udara: {data['kelembapan_udara']} %")
    print(f" Kelembapan Tanah: {data['kelembapan_tanah']} %")
    print(f" PM2.5         : {data['pm25']} µg/m³")
    print("-" * 40)
    time.sleep(5)

```

Output:

```

[2026-05-09 15:53:23] Data terkirim ke topic 'sensor5':
Lokasi      : Lokasi D
Suhu        : 28.57 °C
Cahaya      : 5736 fc
Kelembapan Udara: 82 %
Kelembapan Tanah: 54 %
PM2.5       : 36 µg/m³
=====

```

4.4 Menyimpan ke Database (Subscribe)

Setelah data yang dikirim oleh publikasi tersimpan di broker, subscribe berperan untuk mengambil data tersebut untuk disimpan di database.

```

import json
import os
import paho.mqtt.client as mqtt
from pymongo import MongoClient
from datetime import datetime
from dotenv import load_dotenv
from koneksi import get_mqtt_client, get_mongo_client

load_dotenv()

# --- CONFIG ---
MQTT_BROKER = os.getenv("MQTT_BROKER")
MQTT_PORT = int(os.getenv("MQTT_PORT", 34423))

```



```

MQTT_USER = os.getenv("MQTT_USER")
MQTT_PASS = os.getenv("MQTT_PASS")
MQTT_TOPIC = "sensor5"

mqtt_client = get_mqtt_client()
mongo_client = get_mongo_client()
db = mongo_client["kelompok5_ZAIDAN_HABIBI"]

col_data = db["sensor_tomat"]
col_alert = db["alert"]

# CONNECT CALLBACK
def on_connect(client,rc):
    if rc == 0:
        print(f"[INFO] MQTT CONNECTED ke {MQTT_BROKER}")
        client.subscribe(MQTT_TOPIC)
        print(f"[INFO] Subscribed ke topik: {MQTT_TOPIC}")
    else:
        print("[ERROR] Connect gagal, code:", rc)

# MESSAGE CALLBACK
def on_message(msg):
    try:
        payload = msg.payload.decode()
        data = json.loads(payload)

        # Tambahkan timestamp jika tidak ada
        if "timestamp" not in data:
            data["timestamp"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        # 1. Simpan semua data ke MongoDB
        col_data.insert_one(data)
        print(f"[DB] Data tersimpan dari {data.get('lokasi')}")

        # --- LOGIKA ALERT SESUAI PARAMETER ZAIDAN ---
        alerts = []

        # A. Kelembapan Tanah (Buruk: <40% atau >80%)
        t_moisture = data.get("kelembapan_tanah", 0)
        if t_moisture < 40:
            alerts.append(f"Tanah terlalu kering! ({t_moisture}%)")
        elif t_moisture > 80:
            alerts.append(f"Tanah terlalu basah! ({t_moisture}%)")

```

```

# B. Udara PM2.5 (Buruk: > 150)
pm25 = data.get("pm25", 0)
if pm25 > 150:
    alerts.append(f"Polusi udara berbahaya! (PM2.5: {pm25})")

# C. Cahaya fc (Buruk: < 500 atau > 8000)
cahaya = data.get("cahaya", 0)
if cahaya < 500:
    alerts.append(f"Cahaya terlalu redup! ({cahaya} fc)")
elif cahaya > 8000:
    alerts.append(f"Cahaya terlalu terik! ({cahaya} fc)")

# D. Suhu (Buruk: < 15 atau > 32)
suhu = data.get("suhu", 0)
if suhu < 15:
    alerts.append(f"Suhu terlalu dingin! ({suhu} C)")
elif suhu > 32:
    alerts.append(f"Suhu terlalu panas! ({suhu} C)")

# 2. Jika ada alert, simpan ke koleksi 'alert'
if alerts:
    alert_doc = {
        "lokasi": data.get("lokasi"),
        "timestamp": data.get("timestamp"),
        "nilai_sensor": {
            "suhu": suhu,
            "cahaya": cahaya,
            "tanah": t_moisture,
            "pm25": pm25
        },
        "pesan_alert": alerts
    }
    col_alert.insert_one(alert_doc)
    print(f"\n!!! ALERT TERDETEKSI !!!")
    for a in alerts:
        print(f" -> {a}")
    print("=====\n")

except Exception as e:
    print("[ERROR] Gagal proses data:", e)

# MQTT SETUP
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1)
client.username_pw_set(MQTT_USER, MQTT_PASS)

```

```

client.on_connect = on_connect
client.on_message = on_message

# RUN
try:
    print(f"[INFO] Connecting Subscriber to Broker {MQTT_BROKER}:{MQTT_PORT}...")
    client.connect(MQTT_BROKER, MQTT_PORT, 60)
    client.loop_forever()
except Exception as e:
    print("[ERROR] Sub mati:", e)

```

Output:

```

_id: ObjectId('69fbecd867d007e2161f0bc5')
lokasi : "Lokasi B"
suhu : 25.88
cahaya : 16
kelembapan_udara : 37
kelembapan_tanah : 1
pm25 : 13
timestamp : "2026-05-07 08:37:28"

```

4.5 Membuat Analisis

Analisis berguna untuk mengolah data sensor yang tersimpan di database agar dapat diketahui kondisi kualitas dari 24 jam terakhir. Fungsinya adalah agar user dapat mengetahui kondisi dari tomat mereka, apakah cahayanya terlalu terang dan lain sebagainya.

```

import os
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime, timedelta
from pymongo import MongoClient
from dotenv import load_dotenv

load_dotenv()

# =====
# KONEKSI DATABASE
# =====
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]

```

```

collection = db["sensor_tomat"]

# =====
# AMBIL DATA 24 JAM TERAKHIR
# =====
waktu_sekarang = datetime.now()
waktu_24_jam = waktu_sekarang - timedelta(hours=24)

pipeline = [
    {
        "$addFields": {
            "timestamp_dt": {
                "$dateFromString": {
                    "dateString": "$timestamp",
                    "format": "%Y-%m-%d %H:%M:%S"
                }
            }
        }
    },
    {
        "$match": {
            "timestamp_dt": {"$gte": waktu_24_jam}
        }
    },
    {
        "$group": {
            "_id": {
                "lokasi": "$lokasi",
                "jam": {
                    "$dateToString": {
                        "format": "%Y-%m-%d %H:00",
                        "date": "$timestamp_dt"
                    }
                }
            },
            "avg_pm25": {"$avg": "$pm25"}
        }
    },
    {
        "$sort": {"_id.jam": 1}
    }
]

# =====

```

```

# EKSEKUSI AGGREGATE
# =====
data = list(collection.aggregate(pipeline))

if not data:
    print("Tidak ada data dalam 24 jam terakhir.")
    exit()

# =====
# UBAH KE DATAFRAME
# =====
df = pd.DataFrame(data)

# Pecah kolom _id
df["lokasi"] = df["_id"].apply(lambda x: x["lokasi"])
df["jam"] = df["_id"].apply(lambda x: x["jam"])

df = df.drop(columns=["_id"])

# =====
# SIMPAN CSV
# =====
df.to_csv("data_24jam.csv", index=False, sep=";")
print("Data berhasil disimpan ke data_24jam.csv")

# =====
# PLOT GRAFIK
# =====
plt.figure()

for lokasi in df["lokasi"].unique():
    subset = df[df["lokasi"] == lokasi]
    plt.plot(subset["jam"], subset["avg_pm25"], label=lokasi)

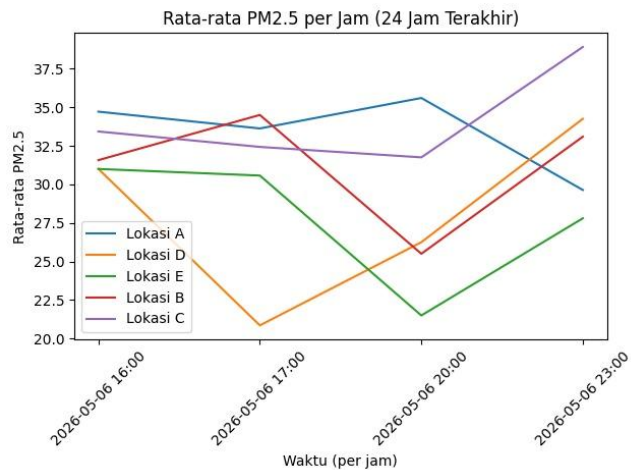
plt.xticks(rotation=45)
plt.xlabel("Waktu (per jam)")
plt.ylabel("Rata-rata PM2.5")
plt.title("Rata-rata PM2.5 per Jam (24 Jam Terakhir)")
plt.legend()

plt.tight_layout()
plt.savefig("dashboard_udara.png")

print("Grafik berhasil disimpan sebagai dashboard_udara.png")

```

Output:



4.6 Kesimpulan

Berdasarkan proyek yang kami kembangkan dapat disimpulkan bahwa program ini sangat berguna untuk user agar mereka dapat mengawasi tanaman mereka tanpa harus berada di lokasi langsung, sehingga user bisa lebih fleksibel. Sistem ini memanfaatkan teknologi IoT untuk menghubungkan sensor-sensor tanaman seperti sensor cahaya, kelembaban tanah, dan kualitas udara agar dapat mengirimkan data secara otomatis dan real-time. Protokol MQTT digunakan sebagai media komunikasi antara publisher dan subscriber karena ringan dan cepat untuk pertukaran data sensor. Data yang diterima kemudian disimpan ke MongoDB untuk proses monitoring, analisis, dan visualisasi dashboard sehingga kondisi tanaman tomat dapat dipantau dengan lebih mudah, efisien, dan responsif terhadap perubahan lingkungan.

BAB V

PENUTUP

Berdasarkan seluruh praktikum yang telah dilakukan, dapat disimpulkan bahwa MongoDB dan Python sangat efektif digunakan dalam pengolahan, monitoring, dan analisis data secara real-time maupun berbasis laporan. Pada tugas terstruktur aplikasi produksi, dipelajari bagaimana membangun aplikasi CLI (aplikasi yang dijalankan dan digunakan melalui terminal) yang mampu melakukan input data, query, agregasi, ekspor laporan CSV (format file yang digunakan untuk menyimpan data berbentuk tabel dalam bentuk teks), logging, serta penggunaan environment variable untuk keamanan koneksi database. Pada studi kasus monitoring suhu mesin, dipelajari penerapan filter waktu, aggregation pipeline, deteksi kondisi abnormal, dan logging alarm untuk kebutuhan monitoring industri. Selanjutnya, pada proyek monitoring tanaman tomat berbasis IoT, sistem berhasil mengintegrasikan sensor, protokol MQTT, MongoDB, dan dashboard analisis sehingga data lingkungan tanaman dapat dipantau secara real-time dan lebih efisien. Dari seluruh materi tersebut dapat dipahami bahwa integrasi database, analisis data, logging, agregasi, serta komunikasi IoT merupakan keterampilan penting dalam pengembangan sistem monitoring dan pengolahan data di dunia industri maupun pertanian modern.